Gestione Risorse e Deadlock

Deadlock

Ricorda

processi bloccati
in reciproca
attesa della risorsa
dell'altro

Condizioni

*mutua esclusione risorse
seriali
*risorse non preemtive
*richieste bloccanti
(hold and wait)
*attesa circolare

necessarie e sufficenti

Tecniche gestione

detection and recovery

detection:
permette deadlock e rileva

aggiorna Holt

casi e cause deadlock:
* 1risorsa1classe->ciclo
*+risorse1classe->non
completamente riducibile
*1risorsa1proc->esiste knot

recovery: risolve
manualmente/automatica:
*terminazione totale/parziale
(costoso, inconsistenza)
*preemtion
*checkpoit-rollback

prevention/avoidance

definizioni

impedice il deadlock

prevention: elimina
condizioni deadlock con
spooling o allocazione:
*totale (memoria)
risorse non note subito
*gerarchica (tempo)
risorse priorità, allocare:
↑priorità ok
↓priorità rilascia risorse
a priorità≥

avoidance: ritarda
assegnamenti risorse
causano deadlock

alg. banchiere

OS=banchiere, Risorse=capitale,
Processo=cliente, allocazione=prestito

*saldo iniziale (IC)
*n. chienti prefissato (N)
*cliente ha max(necessità)<TOT (ci)
* cliente richiede o restituisce
(quando raggiunge ci)
* prestito<necessità (pi)
(credito resitudo è ni=ci-pi)
* sum(prestitiCliente)<necessità
* restituire in tempo finito

bachiere presta o fa attendere

saldo in cassa, SC, IC-sum_{i=1}^Npi

Stato SAFE: richiesta soddisfacibili
con risorse disponibili più allocate

Stato Unsafe necessario ma
non sufficente deadlock

Garanzia: partendo da stato safe
non si rischia deadlock

nella versinone multi valuta tutti i
parametri sono condizionati (k)

Ostrich algorithm

ignora il problema

Classi di risorse

Cosa sono

istanze(risorse)
stessa classe
sono equivalenti
(soddisfano stessa richiesta)

molteplicità=size(classe)

processo richiede una
istanza (non specifica)
di classe (specifica)

Azioni processo

assegnazione risorse:
*statica, dalla creazione
all'eliminazione
*dinamica, all'occorrenza
processo gia avviato
richiede e rilascia

risorsa:
*seriale, 1proc alla volta (W)
* non seriali, +proc alla volta (R)

richiesta:
*singola, 1risorsa-1classe
*multipla, 1/+risorse-1/+classi
*bloccante, proc. sospeso fino
+(rivalutazione a ogni rilascio)
*non bloccante, proc. continua
se assegnazione fallisce

Risorse preemptable

funzione gestione
sottrae (poi restituisce)
risorsa a processo (si sospende)
prima del dovuto

condizione: stato
risorsa indipendente
utilizzo
o salvabile e riprisinabile

Risorse non preemtable sono
l'opposto (stampante)

Grafo (G) di Holt

Cos'è

bipartito (2tipi di nodi)
risorse/classi e processi

diretto (archi direzionati):
*risorsa->processo
assegnata
*processo->risorsa
richiesta insoddisfacibile

nodi stesso partito
non si connettono

Riducibilità

esistenza nodo proc.
con solo archi entranti

riduzione: riassegnare
sue risorse ad'altri proc.

idea: prima o poi termina

Knot

{max(M)|∀n∈M, R(n)=M}

R(n)=insieme di raggiungibilità
(nodi raggiungibili da n), M⊆G